

Incident Report — Listener Crash (2026-08-18)

Incident Report — Listener Crash (2026-08-18)

Summary

On 18 August 2026 the production Telegram listener stopped working. After roughly 101 minutes of operation it shut down silently, with no error message in the logs. The underlying cause was that the listener was sending so many requests to Telegram that it was effectively stuck in a rate-limiting loop, and this eventually exhausted the process's memory, so the hosting platform killed it.

This was a repeat of an earlier incident on 10 August 2026 involving the same user and the same three channels.

What actually happened

The listener runs on a fixed schedule, checking channels every few seconds. Two problems combined to bring it down.

1. Three permanently dead channels were never flagged as broken

The user had three Telegram channels in their setup that no longer existed (@forex_vipxauusd, @gold_pro_forex_trader_xauusd, @gold_pro_trader_forex_signals0). The system has a safeguard meant to disable a channel automatically after it fails five times in a row. But a quirk in the Telegram library used here meant the "this username doesn't exist" error came back in an unexpected format, so the safeguard never recognised it. The listener kept retrying these three dead channels indefinitely — 134 times during the session — and each retry generated more requests to Telegram.

2. Telegram started rate-limiting the account, and the listener had no way to back off

Because the listener was hammering the dead channels (and its normal polling), Telegram started rejecting requests with "flood wait" errors. The listener had no "back off and pause for a while" behaviour, so it kept firing requests into a throttled session. Each rejected request made the listener wait internally, then try again, and these waited requests piled up faster than they completed. At its worst the listener was hitting 500–725 rate-limit errors per minute for the whole 101-minute run.

The combination — endless retries of broken channels plus no pause on rate-limiting — produced a growing pile of in-flight requests and timers that consumed memory until the process was killed.

What we changed

Three fixes were added to the listener, all now committed and pushed:

1. **Dead channels are now actually disabled.** The error classifier was corrected so it recognises the "username doesn't exist" message and, after five consecutive failures, stops the channel and disables it automatically. The three broken channels will no longer be retried forever.
2. **Pause between attempts to rejoin a channel.** If a channel keeps failing to resolve, the listener now waits 10 minutes before trying again instead of hammering it every few seconds.
3. **Automatic pause during rate-limiting.** If Telegram starts rejecting requests, the listener now stops all polling for 2 minutes to let the account cool down, instead of piling requests onto a throttled session. It only resumes once a full polling cycle completes without any flood errors.

Verification

The code compiles cleanly and all 61 worker tests pass, including 20 new tests written specifically for these fixes. The

changes passed an automated test review and a code review; one issue the reviewer flagged was fixed and covered by a new test.

Deployment

The fixes are committed and pushed to the staging, development, and migration branches on both repositories, but **not yet deployed to production**. Per the project's policy, they should go to staging first and be validated before reaching production.

Open questions / follow-ups

- **Confirm exactly how the process died.** The most likely cause is the out-of-memory kill described above, but we should check the error-tracking service (Sentry) for a record at the crash time to be certain.
- **Add protection against this class of failure.** The listener currently has no memory limit or health check configured on the hosting platform, so a similar situation could kill it silently again. Adding those would help.
- **Review the affected account's channel list** and confirm the three broken channels are cleaned up.
- Because the fix is not yet in production, the underlying storm condition still exists until it is deployed.

Bottom line

A dead-channel retry bug combined with no rate-limit back-off caused the listener to overload itself and crash. All three layers of the fix are written, tested, and ready to deploy; they just need to go through the staging process and then to production.